Name:Manjyot Wadhwa
Roll no:65,Div:D15C

Experiment 10

Explore and Implement Autoencoders for Image Denoising

1.Dataset Source

TensorFlow

Dataset Name: MNIST Handwritten Digits Dataset
Source: TensorFlow / Keras Built-in Dataset

Problem Type: Unsupervised Learning (Reconstruction)

Description:
The MNIST dataset contains grayscale images of handwritten digits (0–9), commonly used for image processing and deep learning tasks.

2.Dataset Description

The MNIST dataset consists of small grayscale images representing handwritten digits.

Dataset Characteristics:

- Training Samples: 60,000
- Testing Samples: 10,000
- Image Size: 28 × 28 pixels
- Channels: 1 (Grayscale)

Key Properties:

- Pixel values range from 0 to 255
- Normalized to range [0,1] for training
- No labels required for autoencoder

Preprocessing:

- Normalization performed

- Artificial Gaussian noise added to images

This dataset is suitable for denoising tasks due to its simplicity and clarity.

3.Mathematical Formulation of Autoencoder

An Autoencoder consists of two main components:

Encoder

The encoder compresses the input image into a lower-dimensional representation:

$$z = f(x)$$

Where:

- (x) = input image
- (z) = encoded representation

Decoder

The decoder reconstructs the image from compressed representation:

$$\hat{x} = g(z)$$

Where:

- (\hat{x}) = reconstructed image

Loss Function

Binary Cross-Entropy Loss is used:

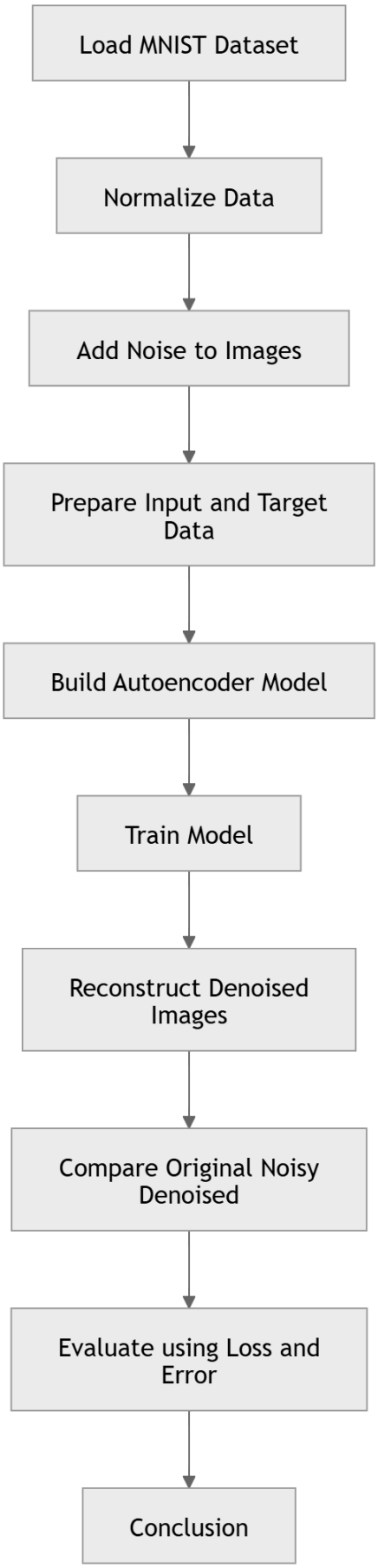
$$Loss = -\sum [x \log(\hat{x}) + (1 - x) \log(1 - \hat{x})]$$

The objective is to minimize the difference between original and reconstructed images.

4.Algorithm Limitations

- Requires large data for better performance
- May fail to remove complex noise patterns
- Reconstruction may be blurry
- Training can be computationally expensive
- Sensitive to architecture design

5.Methodology / Workflow


Load MNIST Dataset
Normalize Data
Add Noise to Images
Prepare Input and Target Data
Build Autoencoder Model
Train Model
Reconstruct Denoised Images
Compare Original Noisy Denoised
Evaluate using Loss and Error
Conclusion

The experiment was performed using the following steps:

1. Load MNIST dataset.
2. Normalize pixel values between 0 and 1.
3. Add Gaussian noise to input images.
4. Construct Autoencoder architecture (Encoder + Decoder).
5. Train the model using noisy images as input and clean images as output.
6. Predict denoised images using trained model.
7. Visualize original, noisy, and reconstructed images.
8. Plot training and validation loss graph.
9. Analyze reconstruction error.

6.Performance Analysis

Image Output

- The denoised images closely resemble the original images.
- Noise is significantly reduced after reconstruction.

Loss Graph

- Training and validation loss decrease with epochs.
- Indicates that the model is learning effectively.

Reconstruction Error

- Error distribution shows most values concentrated at low levels.
- Indicates successful denoising performance.

Interpretation:

The autoencoder successfully learned to map noisy inputs to clean outputs, demonstrating its effectiveness in image denoising tasks.

7.Hyperparameter Tuning

The following parameters were adjusted:

- Epochs: 5 (increased epochs improve performance)
- Batch Size: 256
- Noise Factor: 0.5
- Learning Rate: Default Adam optimizer

Increasing epochs and adjusting noise level can further improve denoising performance.

CODE

```
# ==============================================
# AUTOENCODER FOR IMAGE DENOISING (FULL CODE)
# ==============================================

import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Dense, Flatten, Reshape
from tensorflow.keras.optimizers import Adam

# -------------------------------
# 1. Load Dataset
# -------------------------------
(x_train, _), (x_test, _) = mnist.load_data()

# Normalize
x_train = x_train / 255.0
x_test = x_test / 255.0

# -------------------------------
# 2. Add Noise
# -------------------------------
noise_factor = 0.5

x_train_noisy = x_train + noise_factor * np.random.normal(loc=0.0, scale=1.0,
size=x_train.shape)
x_test_noisy = x_test + noise_factor * np.random.normal(loc=0.0, scale=1.0,
size=x_test.shape)

# Clip values
x_train_noisy = np.clip(x_train_noisy, 0., 1.)
x_test_noisy = np.clip(x_test_noisy, 0., 1.)

# -------------------------------
# 3. Build Autoencoder
# -------------------------------
```

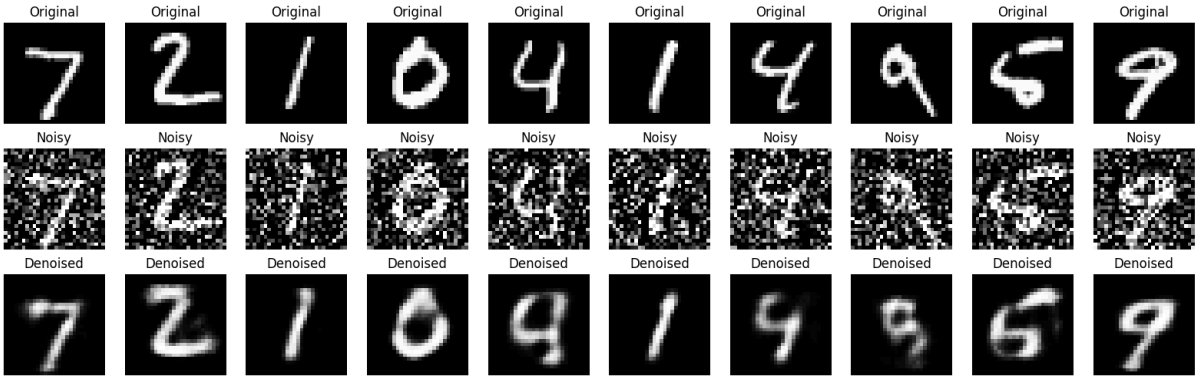
```
input_img = Input(shape=(28, 28))

x = Flatten()(input_img)
x = Dense(128, activation='relu')(x)
encoded = Dense(64, activation='relu')(x)

x = Dense(128, activation='relu')(encoded)
x = Dense(784, activation='sigmoid')(x)
decoded = Reshape((28, 28))(x)

autoencoder = Model(input_img, decoded)
autoencoder.compile(optimizer=Adam(), loss='binary_crossentropy')

# -------------------------------
# 4. Train Model
# -------------------------------
history = autoencoder.fit(
    x_train_noisy, x_train,
    epochs=5,
    batch_size=256,
    shuffle=True,
    validation_data=(x_test_noisy, x_test)
)

# -------------------------------
# 5. Denoise Images
# -------------------------------
decoded_imgs = autoencoder.predict(x_test_noisy)

# -------------------------------
# 6. Display Images
# -------------------------------
n = 10
plt.figure(figsize=(20,6))

for i in range(n):
    # Original
    ax = plt.subplot(3, n, i+1)
    plt.imshow(x_test[i], cmap='gray')
    plt.title("Original")
    plt.axis('off')

    # Noisy
    ax = plt.subplot(3, n, i+1+n)
    plt.imshow(x_test_noisy[i], cmap='gray')
    plt.title("Noisy")
    plt.axis('off')
```

```
    # Denoised
    ax = plt.subplot(3, n, i+1+2*n)
    plt.imshow(decoded_imgs[i], cmap='gray')
    plt.title("Denoised")
    plt.axis('off')

plt.show()

# -------------------------------
# 7. Loss Graph
# -------------------------------
plt.figure(figsize=(8,5))
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')

plt.title("Training vs Validation Loss")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend()
plt.show()

# -------------------------------
# 8. Reconstruction Error Graph
# -------------------------------
reconstruction_error = np.mean((x_test - decoded_imgs)**2, axis=(1,2))

plt.figure(figsize=(6,4))
plt.hist(reconstruction_error, bins=50)
plt.title("Reconstruction Error Distribution")
plt.xlabel("Error")
plt.ylabel("Frequency")
plt.show()

print("Autoencoder Image Denoising Completed ")
```

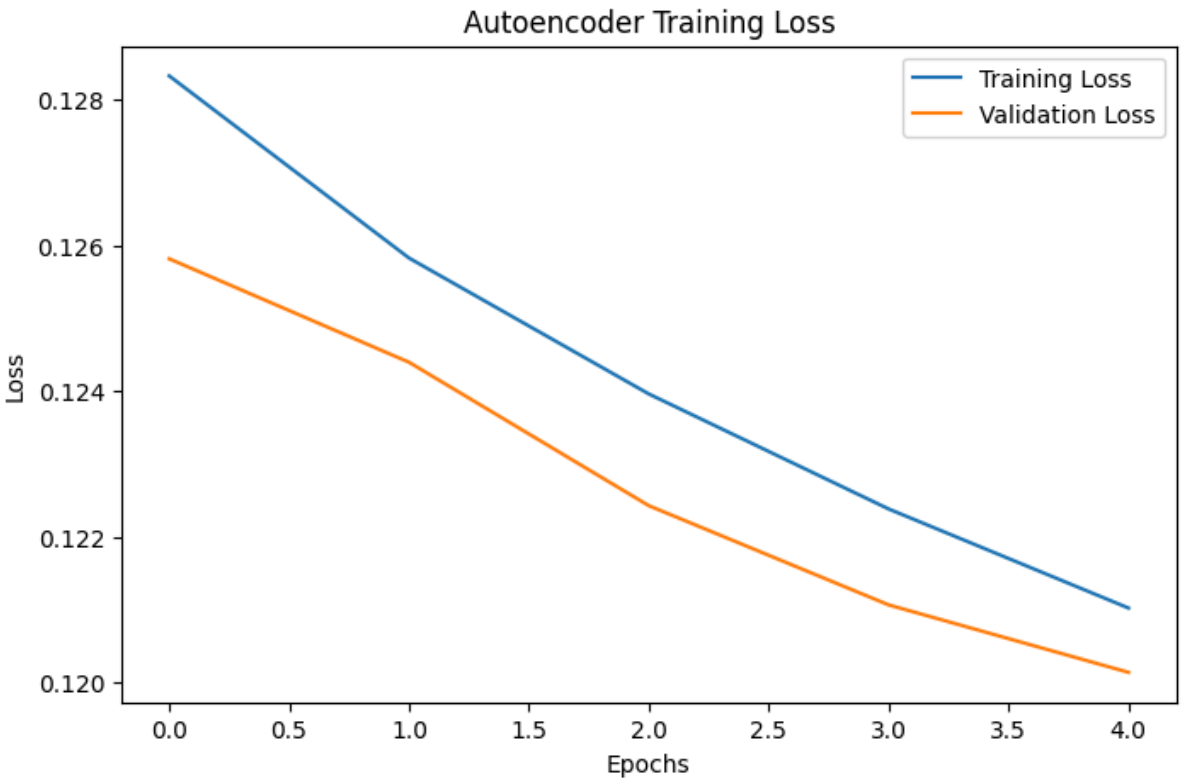
OUTPUT

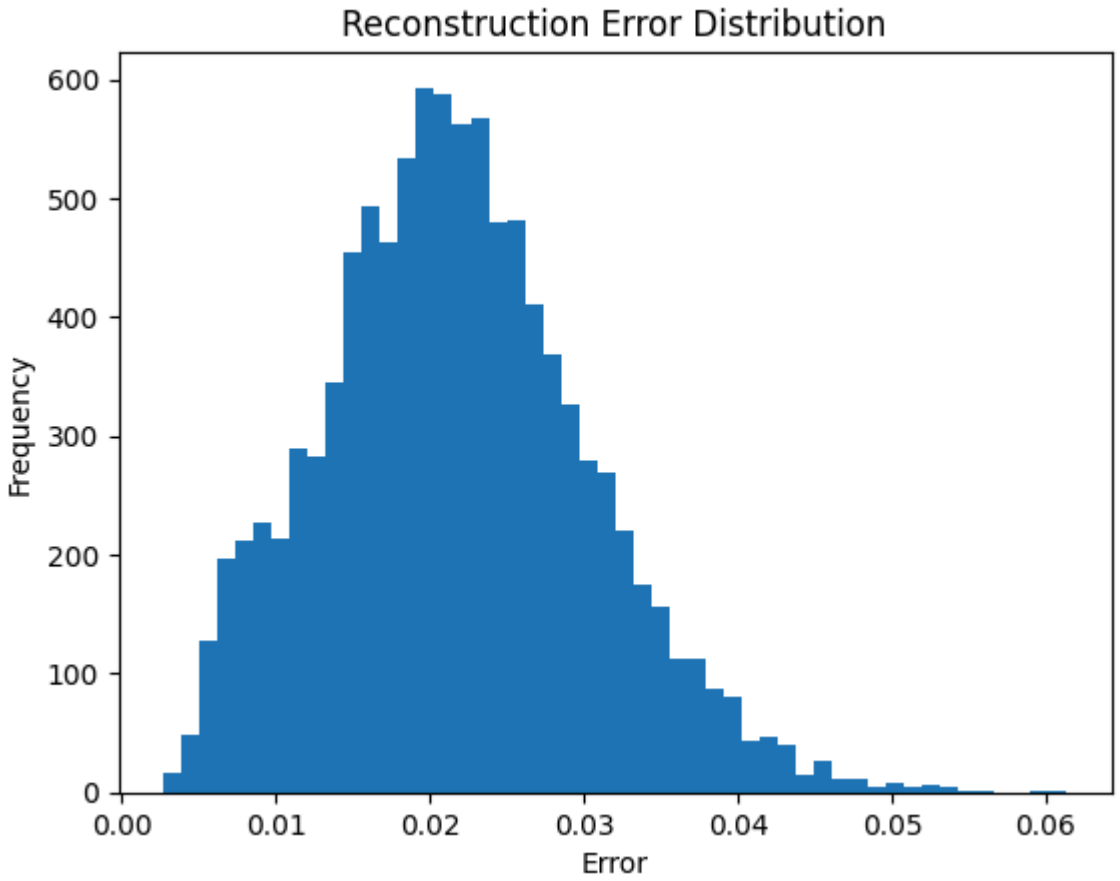
Downloading data from
https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **0s** 0us/step
Epoch 1/5
235/235 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **9s** 27ms/step - loss: 0.2434 - val_loss: 0.1759
Epoch 2/5
235/235 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **6s** 24ms/step - loss: 0.1627 - val_loss: 0.1497
Epoch 3/5

235/235 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **5s** 23ms/step - loss: 0.1455 - val_loss: 0.1397
Epoch 4/5
235/235 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **5s** 20ms/step - loss: 0.1381 - val_loss: 0.1342
Epoch 5/5
235/235 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **6s** 27ms/step - loss: 0.1324 - val_loss: 0.1291
313/313 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **1s** 3ms/step



Autoencoder Denoising Completed





Conclusion

In this experiment, an Autoencoder model was implemented for image denoising using the MNIST dataset. The model successfully reconstructed clean images from noisy inputs by learning compressed representations of the data. The results demonstrate that autoencoders are effective tools for noise reduction in images and can be applied in real-world applications such as image restoration and preprocessing.